

INTERACTIVE GRAPHICS — FINAL PROJECT

ECHOES IN THE DARK

A first-person procedural horror experience built from scratch in WebGL / Three.js

Team Trio Monnezza

Davide Timperi · 1950722

Alexandru Vivian Pita · 1948533

Federico Mendiola · 1986026

OVERVIEW

Trapped, in the dark, not alone

The player wakes up inside a dark, procedurally-furnished building. A single flashlight with a draining battery is the only source of light — and a hierarchically-animated, AI-driven monster is somewhere in the building, hunting.

Goal: find the hidden golden key and reach the exit door before the monster finds you first.

3

Difficulty levels

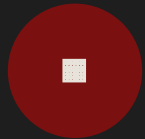
100%

Procedural animation — zero imported clips

1

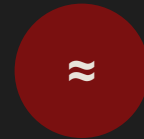
Light source standing between you and the dark

Built on WebGL — nothing else does the heavy lifting



Three.js r160

Rendering engine only — scene graph, cameras, lights, loaders. All game logic, animation and collision is original code.



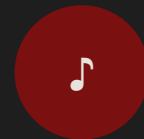
tween.js 21.0

Generic interpolation engine. Every curve, timing and hierarchical rig motion is hand-authored by the team.



GLTFLoader / FBXLoader

Official Three.js example modules, used strictly for importing static (non-animated) meshes.



Web Audio API

Wrapped in a custom AudioSystem for positional 3D sound and dynamic music cross-fading.

Four systems the player is always juggling



Stamina

Sprint (Space) drains stamina; running dry forces a slow walk at the worst possible moment.



Flashlight battery

Drains over time. Dimmer, narrower, flickering beam as it dies — recharge with battery pickups.



The Monster

Patrols and hunts via line-of-sight. Cannot be fought — only evaded, outrun, or shut out.



Doors

Open/close with E. Physical barrier and a line-of-sight blocker for the monster.

Levels are grids, not hand-placed geometry



Grid-based construction

Each of the three difficulty maps extends a shared MapBase that reads a grid of typed cells and instances wall, floor, ceiling, door and pillar tiles procedurally from it.



Three material themes

Wood/wallpaper, plaster/tile and stone/brick/concrete distinguish the three difficulty levels visually.



Proportional furnishing

InteriorAssetManager.js preloads the prop library; PropScale.js gives every prop a real-world-accurate scale relative to the player's eye height.

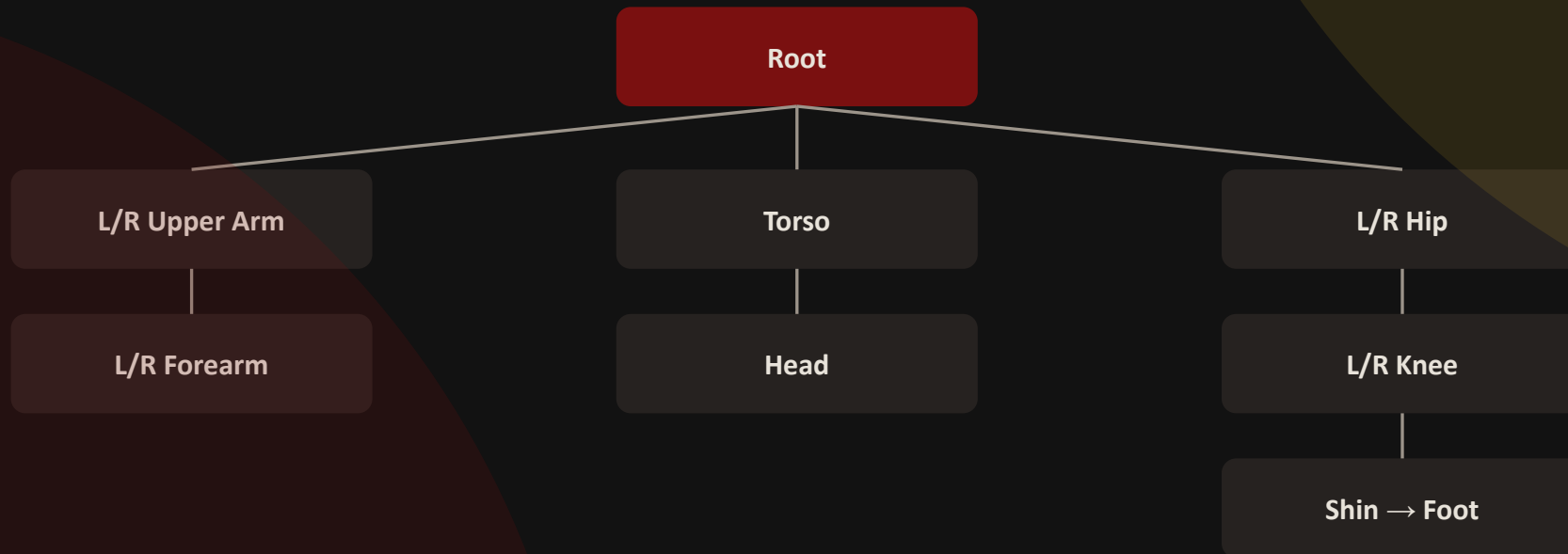


Collision built inline

The same pass that instances a wall or door tile also registers its own axis-aligned bounding box.

The monster: a hand-built Object3D rig

No rigged mesh is imported — every joint is an Object3D parent/child relationship assembled in code from primitive geometry.



Four states, one continuous tween.js state machine



Walk cycle

Opposite-phase sine waves on shoulders, hips and the new knees (double hip frequency, offset between legs), plus a torso bob and a forward lean while chasing.



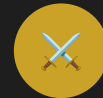
Idle

Slow breathing, broken up by unpredictable random “spasms” so the loop never looks mechanically perfect.



Notice (shock)

A sharp startle — head snap, torso recoil, arms jerking up — fired the exact frame the AI first detects the player.



Attack

Anticipation (wind-up) → diagonal claw strike → recoil, chained tween-to-tween into one continuous curve.

THE GOLDEN RULE One owner per property.

The lean writes to the torso's local rotation, never the root — the root's rotation is exclusively driven by `lookAt()`, so writing both causes conflicting orientations.

Reactive, not scripted — sight, memory and persistence



Detection

The instant distance-to-player first drops below the aggro radius, a one-shot “notice” event fires — not a per-frame check.



Line-of-sight gated attack

A raycast against the level's own collision boxes must confirm a clear line before the monster lunges — closing a door genuinely blocks the kill.



Steering

A fan of forward probe rays picks the least-obstructed heading each frame, with a stuck-detection fallback so the AI never gets wedged in geometry.



Persistence

If the player stays out of range too long, or the monster is stuck without sight of them, it relocates near the player — never purely avoidable by distance.



Door-aware

The monster can open an unlocked closed door itself while patrolling toward the player.

No physics engine — custom AABB, built for a first-person feel



Axis-aligned bounding boxes

THREE.Box3 volumes built once per level, directly while each wall / door / prop tile is instantiated.



Per-axis sliding

X and Z are tested and applied independently, producing a smooth glide along a wall on an oblique approach instead of a hard stop.



Shared by player & monster

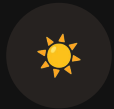
The monster resolves its own movement against the exact same bounding-box set, including a forward probe-ray fan for steering.



Dynamic door boxes

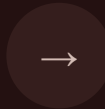
A door's box empties once it is visibly ajar and snaps solid again the moment it swings shut — both characters are physically blocked by a closed door.

Four light types, three material themes



Ambient

Low-level fill so no surface is ever pure black



Directional

Faint global key light



Spot

The player's flashlight — battery-driven intensity, angle & range



Point

Flashlight fill light + per-room accents, some flickering

Materials: THREE.MeshPhongMaterial + diffuse / normal / specular maps

Normal maps add surface relief; specular maps vary shininess per material (e.g. a wet floor highlights differently than a dry one). Three distinct sets — wood/wallpaper, plaster/tile, stone/brick/concrete — give each difficulty level its own visual identity. Procedural objects (key, batteries) use PBR MeshStandardMaterial instead — both lighting models coexist, each where it fits best.

A mix that reacts to distance, not just to events



Positional 3D audio

The monster's breathing, door creaks and slams are emitted from their real 3D position, so the player can judge distance and direction by ear alone.



Dynamic music

The background track smoothly cross-fades between an “ambience” mix and a more intense “pursuit” mix depending on the monster's proximity.



Movement-linked footsteps

Footstep audio tracks the player's walking/sprinting state in real time.



One-shot UI/SFX pool

Pickups, menu interactions and door impacts each fire a dedicated, precisely-timed sound cue.

Two endings, mirrored in code



DEATH

1

Blood splatter

2

Fade to black

3

Game Over
text fades in

Procedural blood-splatter shapes — organic blobs from randomised irregular border-radius values, a radial-gradient wet highlight, and CSS box-shadow droplets. No two deaths render identical marks.



VICTORY

1

Door opens &
shrinks

2

Light bloom +
sparks

3

Escape text
fades in

The exit door animates open. A warm light overlay blooms across the screen with soft, rising, fading particle sparks — a deliberate visual mirror of the death sequence, escalating instead of dimming.

An LLM as level-decorator, not as a coder



Scope

The team used Claude as a room-decoration assistant, given the already-built structural maps as context.



Constraints

Place only existing prop models; respect real-world proportions; keep walking space clear; avoid overlapping collisions; keep rooms thematically coherent.



Output

Placement data only, consumed by InteriorAssetManager.js exactly like any other prop instance.



Boundary

The assistant did not write or modify any game logic, animation, AI, or rendering code.

Checklist against the course requirements



WebGL / approved library

Three.js — rendering & loaders only



Hierarchical model

Monster: multi-level Object3D parent/child rig



No imported animations

Every motion computed at runtime with tween.js



Lights & textures

Ambient / directional / spot / point + diffuse, normal, specular maps



User interaction

Movement, sprint, flashlight, doors, difficulty, menus



Physics / collision

Custom AABB with per-axis sliding — no physics engine

Getting started & controls

PLAY ONLINE

Open the GitHub Pages link in any modern desktop browser (Chrome / Firefox / Edge). No install needed.

RUN LOCALLY

Clone the repo, then serve it over HTTP (e.g. `python -m http.server 8000`) — assets are fetched, so `file://` will not work.

W A S D

Move

Mouse

Look around

Space

Sprint (hold)

E

Interact — doors, key, batteries

G

Debug safe-mode toggle (off by default)

Objective & difficulty levels

Find the hidden golden key, then reach the marked exit door. Avoid the monster — if it catches you, the run ends.

EASY

Wood / wallpaper interior

1 battery

MEDIUM

Plaster / tile interior

2 batteries

HARD

Stone / brick / concrete interior

3 batteries

READY TO SURVIVE THE NIGHT?

Thank you.

sapienzainteractivegraphicscourse.github.io/final-project-triomonnezza

Team Trio Monnezza — Davide Timperi · Alexandru Vivian Pita · Federico Mendiola